Discovery Executive Summary

Project: disocovery-0123 · **Generated:** 24/07/2026, 14:28:27

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

---

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service/Repository layers (H2/H3), Direct SQL/ORM in controllers (H6), Domain boundary & shared-DB coupling (H8/H9), and dual-stack logic duplication (H10).

EXECUTIVE SUMMARY

Klearcom is a multi-layer Voice/Telecom QA monolith (Laravel 12 API, React 19 SPA, Node Express `dev-api`) with nominal Discovery/Connect module folders that contain only `AGENTS.md` stubs — real logic lives in fat HTTP handlers and two shared services. Controllers and Express routes call Eloquent/in-memory stores directly (no repository layer), duplicate reachability/`buildTree`/KPI math across PHP and Node, and cross-read both domains from Dashboard and Legacy report paths. The React layer has a thin `api/client.ts` but hard-codes module paths inside page components, mixes a legacy class poller with modern hooks, and lacks error boundaries. Dominant risk is change amplification and hidden coupling: a reachability formula or IVR tree shape change requires coordinated edits in Laravel controllers, `RealTimeTestService`, `dev-api`, and both SPA pages.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	62 LOC	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 Eloquent access points	HIGH RISK
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	35+ (0 repositories)	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2	MODERATE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	~29% kept out of controllers	HIGH RISK
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest server.js 276 lines)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	80% (4/5 tables cross-read)	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	74 LOC	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (ConnectPage 222, DiscoveryPage 176)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2 levels; focused Zustand store	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2	MODERATE
H10	Dual-stack domain	Duplicated workflows across Laravel	0	1–3	>3	5 duplicated workflows	HIGH RISK

	duplication (additional)	↔ Node (target 0)					
H11	Empty bounded-context shells (additional)	Module dirs with zero impl classes (target 0)	0	1–2	>2	2 empty module shells	MODERATE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 Missing Service Layer	Extract Dashboard/Discovery/Connect application services; thin controllers to HTTP only	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add MariaDB repository interfaces/impls; stop Eloquent from controllers/services entrypoints	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Replace `LegacyDataMapper` extract() with DTOs; move `buildTree` into Discovery domain service	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Relocate all Eloquent builders out of controllers into repositories via services	HIGH RISK	CRITICAL
H8 Domain Boundary Violations	Enforce Discovery/Connect ownership; Dashboard/Legacy consume published APIs/ACL	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Declare per-table ownership; eliminate cross-domain Model:: reads	HIGH RISK	CRITICAL
F5 Legacy Component Patterns	Migrate class poller to hooks; add ErrorBoundary; standardize TSX	MODERATE	MEDIUM
H10 Dual-stack duplication	Consolidate domain logic onto Laravel; shrink/proxy `dev-api`	HIGH RISK	CRITICAL
H11 Empty module shells	Relocate domain classes into `Modules/*` packages; enforce import boundaries	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers and Express handlers become thin HTTP adapters; KPI/reachability/tree workflows live once in application/domain services and are unit-testable without a database.
- Repository interfaces isolate MariaDB/Mongo persistence, enabling deterministic tests and safer schema evolution.
- Discovery and Connect evolve as real bounded contexts with owned models/tables and anti-corruption edges for Dashboard/Legacy.
- Dual-stack drift disappears when Node stops re-implementing domain rules; frontend module API facades + ErrorBoundary reduce path churn and uncaught UI failures.
- Empty module folders become enforceable packages, preparing the monolith for independent extraction of Discovery or Connect.